Quest #77

Registration

EXEQUT

Hiring Quest – Java Tech Lead @ ExeQut

Challenge-based hiring quest with structured evaluation and real project outcomes.

Status: Registration **Registration:** March 19, 2026 **Submission:** March 26, 2026

META INFO

CATEGORY	STATUS
Registration	Registration
REGISTRATION DEADLINE	**SUBMISSION DEADLINE**
March 19, 2026	March 26, 2026

PRIZE

Top performers get hired with a paid contract and the opportunity to work on real-world projects.

QUEST BRIEF

Cart Checkout + Mock Payment System

Single Service | Java / Spring Boot

Business Context

You are designing the backend for a small e-commerce product.

The system allows users to:

1. Create a cart
2. Add items
3. Checkout
4. Pay for their order

This system is **not expected to scale**

However:

1. It handles real money
2. Order state must always be correct
3. No double payments
4. No invalid state transitions
5. Webhooks may arrive twice

You are the Tech Lead responsible for:

1. Defining the domain model and designing clear boundaries inside a single service
2. Designing the order state machine and protecting business invariants
3. Ensuring data consistency and correctness in all state transitions
4. Handling payment safely, including webhook processing and idempotency
5. Defining a safe payment handling strategy that prevents double charges
6. Designing failure handling for duplicates, retries, and race conditions
7. Making conscious architectural trade-offs (simplicity vs robustness)
8. Enabling future extensibility (refunds, partial payments, cancellations) without breaking core invariants

---

User Flows

Flow 1 — Happy Path

1. Create cart
2. Add items
3. Checkout → Order created
4. Start payment
5. Mock provider confirms payment (webhook)
6. Order becomes PAID

Flow 2 — Payment Failure + Retry

1. Start payment
2. Provider sends FAILED
3. Order becomes PAYMENT_FAILED
4. Start payment again
5. Provider confirms → Order becomes PAID

Flow 3 — Duplicate Event Handling

1. Provider may send the same webhook twice
2. System must remain correct
3. No duplicate transitions
4. No corrupted state

Functional Requirements

Cart

Endpoints:

1. POST /carts → create cart
2. POST /carts/{cartId}/items → add item (productId, quantity, price)

Cart rules:

1. Items can be modified before checkout
2. Checkout locks the cart

Checkout / Order

Endpoint:

1. POST /carts/{cartId}/checkout

Behavior:

1. Creates an Order from cart
2. Locks cart
3. Initializes order state

Required Order States

You must define and implement a state machine.
Minimum states:

1. CREATED
2. PENDING_PAYMENT
3. PAYMENT_FAILED
4. PAID

(Optional: CANCELLED if you can support it cleanly.)

You must enforce valid transitions.

🌊 Payment

Endpoints:

1. POST /orders/{orderId}/payment/start
2. POST /payments/webhook

Required Behavior

When starting payment:

1. Create a payment intent / attempt
2. Order moves to PENDING_PAYMENT
3. Must prevent duplicate active payments

When webhook is received:

1. Update order state safely
2. Handle duplicate webhook calls idempotently

Mock Payment Provider (Simple Requirement)

You must implement a minimal mock provider inside the same project that:

1. Simulates payment start
2. Allows triggering a payment result (CONFIRMED / FAILED)
3. Calls your /payments/webhook endpoint

No need for complex simulation.
Just enough to demonstrate full flow.

Testing Requirements

Minimum:

1. 1 unit test validating domain state transition

Optional but strong:

1. 1 integration test covering happy path
2. Test for duplicate webhook

Deliverables

Your submission must include:

1. Complete Java / Spring Boot codebase
2. Clear domain model & order state machine
3. Payment handling (including idempotency & webhook logic)
4. Mock payment provider
5. Required endpoints implemented
6. At least 1 unit test for state transitions
7. README (setup + assumptions + key decisions)

10–15 min Video (Required)

Cover:

1. Architecture & key decisions
2. State machine & payment flow
3. Webhook handling & idempotency
4. Edge cases (failures, retries, duplicates)
5. Trade-offs & improvements

The technical review session will focus on your submitted solution, so make sure you fully understand and can clearly walk us through your implementation.

Evaluation Criteria

Area & Weight

State machine & invariants: 25%

Idempotency & payment safety: 25%

Code clarity & structure: 25%

Testing: 20%

Documentation clarity: 5%

QUEST SNAPSHOT

STATUS
Registration

REGISTRATION
March 19, 2026

SUBMISSION
March 26, 2026

Register Now

Need help? Contact us on WhatsApp at **01558405326** or join our **community**.

Quest #77